
Anonymous function:

- It is a nameless function created in a single line.
- It is declared without a name using the lambda keyword.
- It contains n number of parameters which are separated by comma(,) .
- It contains only one expression.
- Parameters and expression is separated by colon(:) .
- It returns a lambda function address, that needs to be stored in a local variable.
- With the help of that variable we pass the values to the parameters.

Syntax: lambda Parameters : Expression

Example 1 – Even Number Check

```
is_even = lambda num: num % 2 == 0
print(is_even(10)) # True
print(is_even(11)) # False
```

Example 2 – Addition

```
res = lambda a, b: a + b
print(res(10, 4)) # 14
print(res(10)) # TypeError
```

Example 3 – Square of Number

```
var = lambda n : n **2
print(var(int(input("Enter: "))))
Input: Enter: 3
Output: 9
```

Example 4 – Divisible by 3 and 4

```
var = lambda n : n % 3 == 0 and n % 4 == 0
print(var(int(input("Enter: "))))
Input: Enter: 12
Output: True
```

Built-in Higher Order Functions

A function that works with other functions is called a Higher Order Function.

1. Filter() :

- It is a built-in function in Python.
- It is used to filter data based on a condition.
- The function returns only the elements that satisfy the condition.
- If the condition is True → element is accepted
- If the condition is False → element is rejected

Syntax: filter(function, iterable)

- function → condition checking function(user-defined / lambda)
- iterable → list, tuple, etc.
- filter() returns a filter object
- To see the result we need to convert it into a list.
- **list(filter(function, iterable))**

Example 1 : Filtering even numbers Using user-defined function

```
def is_even(num):  
    if num % 2 == 0:  
        return True  
    else:  
        return False  
  
list = [2, 4, 45, 67, 98, 11, 10]  
print(list(filter(is_even, list)))
```

Output: [2, 4, 98, 10]

Example 2 : Filtering even numbers Using Lambda function

```
list = [2, 4, 45 ,67 ,98 ,11 ,10]
print(list(filter(lambda num : num % 2 == 0, list)))
```

Output : [2, 4, 98, 10]

Example 3 : Filtering even numbers by Passing List Directly

```
print(list(filter(lambda num : num % 2 == 0, [2, 42, 56, 67, 81, 41, 11] )))
```

Output : [2, 42, 56]

Example 4 : Filter Strings with Length > 3

```
words = ['hi', 'hello', 'cat', 'python', 'go']

longwords = list(filter(lambda w: len(w) > 3, words))
print(longwords)
```

Output : ['hello', 'python']

Example 5 : Filter Mobiles with Price > 30000

```
mobiles = [
{'name' : 'iphone' , 'price' : 80000},
{'name' : 'oneplus' , 'price' : 35000},
{'name' : 'realme' , 'price' : 20000},
{'name' : 'samsung' , 'price' : 45000}
]
result = list(filter(lambda x: x['price'] > 30000, mobiles))
print(result)
```

Output:

```
[{'name' : 'iphone' , 'price' : 80000},
{'name' : 'oneplus' , 'price' : 35000},
{'name' : 'samsung' , 'price' : 45000} ]
```

2. Map() :

- It is a built-in function in Python.
- It is used to apply a same function to every element in an iterable.

Syntax: map(function, iterable)

- function → condition checking function(user-defined / lambda)
- iterable → list, tuple, etc.
- map() returns a map object.
- To see the result we need to convert it into a list.
- **list(map(function, iterable))**

Example 1 : Doubling the List Elements Using Normal Function

```
def square(num):  
    return num + num
```

```
list = [23, 4, 6, 7, 2,9]  
print(list(map(square, list)))
```

Output : [46, 8, 12, 14, 4, 18]

Example 2 : Doubling the List Elements Using Lambda function

```
print(list (map (lambda n : n + n, [2, 4, 6, 7] ) ) )
```

Output : [4, 8, 12, 14]

Example 3 : Add 10 to Each Element

```
n = [5, 10, 15, 20 ]  
res = list ( map ( lambda x : x + 10, n) )  
print(res)
```

Output : [15, 20, 25, 30]

Example 4 : Add Two Lists

```
a = [1,2,3]
b = [4,5,6]

res = list(map(lambda x , y : x + y, a, b))
print(res)
```

Output : [5, 7, 9]

Example 5 : Convert Names to Uppercase

```
names = ["ram" , "ravi" , "anu"]

res = map(lambda x: x.upper(), names)
print(list(res))
```

Output : ['RAM' , 'RAVI' , 'ANU']

Example 6 : Find Length of Each Word

```
words = ["pen" , "pencil" , "book" , "eraser"]

result = map(len, words)
print(list(result))
```

Output : [3 , 6 , 4 , 6]

Example 7 : Convert Numbers to Strings

```
numbers = [10,20,30]

result = map(str, numbers)
print(list(result))
```

Output : ['10', '20', '30']

3.Reduce()

- reduce() is a function present in the functools module.
- This function iterates through each value in an iterable and reduces them to a single value based on the expression or condition given in the function.
- In Simple terms : takes many values and reduces them into one single value.

Syntax: from functools import reduce
reduce(function, iterable, initializer)

- function → condition checking function(user-defined / lambda)
 - iterable → list, tuple, etc.
 - initializer → optional starting value. It is used as the initial value before the iteration starts.
-

Example 1 – Sum of Elements without Initializer

```
from functools import reduce
```

```
list = [1, 2, 3, 4, 5]  
res = reduce(lambda a ,b : a + b, list )  
print(res)
```

Working: ((((1+2) +3) +4) +5)

1 + 2 = 3

3 + 3 = 6

6 + 4 =10

10 + 5 =15

Output : 15

Example 2 – Sum of Elements with Initializer

```
from functools import reduce

list = [1, 2, 3, 4, 5]
res = reduce(lambda a, b : a + b, list, 5)
print(res)
```

Working: (((((5 + 1) + 2) + 3) + 4) + 5)

5 + 1 = 6
6 + 2 = 8
8 + 3 = 11
11 + 4 = 15
15 + 5 = 20

Output : 20

Example 3 – Product without Initializer

```
from functools import reduce

list = [2, 3, 4]
res = reduce(lambda a,b : a * b, list, 2)
print(res)
```

Working: (((2*2) *3) *4)

2 * 2 = 4
4 * 3 = 12
12 * 4 = 48

Output: 48

Example 4 – Finding Maximum Value without Initializer

```
from functools import reduce

list = [10, 25, 8, 40]
res = reduce(lambda a, b : a if a > b else b, list)
print(res)
```

Output: 40

Example 5 – String Concatenation with Initializer

```
from functools import reduce

words = [ "Python", "is", "easy" ]
res = reduce ( lambda a , b : a + " " + b, words, "Learning")
print(res)
```

Output : Learning Python is easy

Example 6 – Count Total Characters in Words

```
from functools import reduce

words = ["pen", "book", "pencil"]

result = reduce(lambda x , y: x + y, map(len , words))
print(result)
```

Output : 13

Example 7 – Find Longest Word

```
from functools import reduce

words = ["pen", "pencil", "book", "erasers"]

result = reduce(lambda x , y: x if len(x) > len(y) else y, words)
print(result)
```

Output : erasers
